In simple words

James Augustus

In one sentence

An idea for a processor that keeps every number in two forms and turns multiplication into addition. There is no chip yet. What I have are models that run on a computer, and tests.

I started with colour

There are three light channels: red, green and blue. Each one is either on or off, which gives eight colours. Eight states are exactly one digit in the octal number system, a digit from 0 to 7. So every digit is a colour. The digit 5 is red and blue lit together, which is magenta. I call this three bit unit an xbit. The number 360 is written 550 in octal, so it reads magenta, magenta, black.

For me colours are not only a notation. Colours combine as a physical event in real life, and the result appears at the speed of light. A physical event is a natural logic gate for me. I chose the octal system because I want to use that in mathematics, and the number 8 fits very well in every experiment I run. I also have physical designs with real colours, but there is much more to do before those. I found similar work that builds octal logic on eight light colours: Talal, Abu-Elnasr and Elmougy, 2021.

Keeping a number in two forms

To add, I keep the number as digits. To multiply, I split the number into prime factors and write how many times each prime appears. I call this the shadow. Since $360 = 2^3 \times 3^2 \times 5$, its shadow is three 2s, two 3s and one 5.

In the shadow, multiplication turns into addition. Take 12 times 30. Here $12 = 2^2 \times 3$ and $30 = 2 \times 3 \times 5$. I add the exponents: for 2 it is $2 + 1 = 3$, for 3 it is $1 + 1 = 2$, and for 5 it is $0 + 1 = 1$. The result is $2^3 \times 3^2 \times 5 = 360$. Division turns into subtraction, and a square root turns into halving the exponents. Instead of working out the answer digit by digit, I move the exponents.

Calculating like a person

A person does not do 10 times 9 by adding nine tens, they say 90. They turn 21×36 into 7×108. In an exam they leave $3\sqrt{2}$ as it is and do not write a ten digit decimal. I wanted the machine to work like that: it factors, keeps the symbol, and opens it only at the end. Square roots, inverses, sums and factorials are carried as types too, and opened only when a value is needed.

Who finds the shadow

A part called Resolver turns a number into its shadow. I explain it with gears and dials, with no division. For each of the seven small primes (2, 3, 5, 7, 11, 13 and 17) there is a dial that reads the octal digits from left to right and counts how many times its prime appears. What is left over, which none of these primes divide, I call rough, and I keep it as it is. The details are in the Gear model.

What it is and what it is not

There is no silicon. I do not claim any gain in speed or energy. What I have is the idea, working models, 32 test packages and the record of the claims that did not hold. Four of my ideas were tested and failed, and I left them in the journal.

Why I am doing this

This is my hobby and my life. I grew up and studied in the culture of a scientist. In my lab courses I wrote reports and took measurements hating every bit of it, but back then they had a meaning and a purpose. Now, with the technology we have, I will not write like a hobbyist from sixty years ago. I will research a lot and write it out finely, and I will wander off where I need to. My aim is simple: to use in computer language the same tactics we use when we calculate. And to learn something myself whenever I work out an operation.

Where I am stuck

I have questions I am stuck on and places I want to see developed. I would like people to develop them themselves, and I am open to suggestions. My open questions are in section 16 of the Final Form.

Where to go next

If you are curious, read the Gear model first, then the Final Form, then the journal.

James Augustus, 2026. Licensed under CC BY 4.0. https://github.com/JamesAugustus/ctcpu-hunter